MSA-DATB09-03 · NEURAL NETWORKS & DEEP LEARNING FOUNDATIONS

Session 1 / 8

The Perceptron

The first artificial neuron: we compute it by hand, we code it in NumPy, and we find its limits.

Guillaume Saint-Cirgue Albert School · MSc 1 October 2026

Guillaume Saint-Cirgue your instructor

Data Scientist and ML Engineer since 2016

2013 to 2016	Engineering degree, **ENSMM** (Besançon)
2016 to 2017	Data Scientist, **Rolls-Royce**
2017 to 2019	Lead ML Engineer, **GKN Aerospace**
2019	Launched **Machine Learnia** on YouTube: 200,000+ subscribers today
2020	Founded my company and my startup: **30+ AI projects** delivered since
2023	Head of Data, **Airbus**
Today	**ML-PRO** training program: 800 students

To go further

MY YOUTUBE CHANNEL

Machine Learnia

200,000+ subscribers. Machine learning and deep learning videos, including a neural network coded from scratch in NumPy.

youtube.com/@MachineLearnia

MY EBOOK

Apprendre le Machine Learning en une semaine

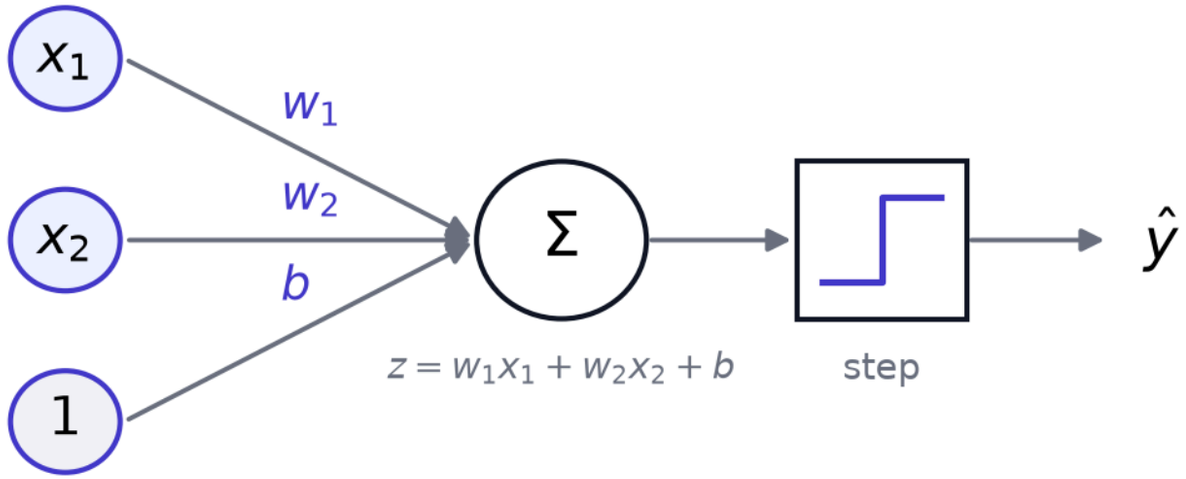
In English: Learn Machine Learning in One Week.

machinelearnia.com/apprendre-le-machine-learning-en-une-semaine

Both resources are in French. Click the cards in your copy of the course to open them.

One artificial neuron

- A biological neuron receives signals, some stronger than others, and **fires** when the total passes a threshold.
- **McCulloch and Pitts (1943)**: a neuron as a weighted sum followed by a threshold.
- **Rosenblatt (1958)**: the **perceptron**, a neuron that **learns** its weights from examples.



Inputs, weights, a sum, a threshold: that is all.

The perceptron model

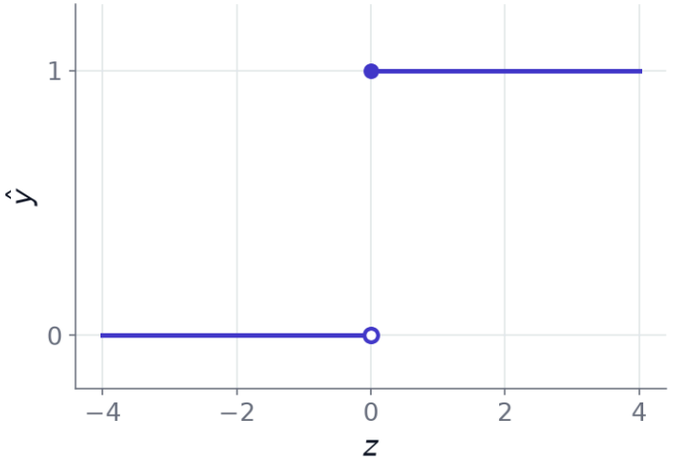
With two inputs x_1, x_2, two weights w_1, w_2 and a bias b:

$$z = w_1 x_1 + w_2 x_2 + b \qquad \hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

With n inputs, in vector form, $x, w \in \mathbb{R}^n$:

$$z = w^\top x + b = \sum_{j=1}^{n} w_j x_j + b, \qquad \hat{y} = \text{step}(z)$$

z is the **score**, $\hat{y}$ the **prediction** (0 or 1), y the true label.



The step function: 1 from $z = 0$ on.

Exercise 1 · A perceptron computes AND

Weights $w = (1, 1)$, bias $b = -1.5$.

x_1	x_2	AND	z	$\hat{y}$
0	0	0		
0	1	0		
1	0	0		
1	1	1		

Exercise 10 min pen and paper

1. Fill in z and $\hat{y}$ for the 4 inputs. Does the perceptron compute AND?
2. Find weights and a bias that compute **OR** (output 1 if at least one input is 1).
3. Bonus: find weights for **NAND** (NOT AND).

Hint: For OR, keep $w = (1, 1)$ and only change b.

Your notes / answers (saved in this browser)

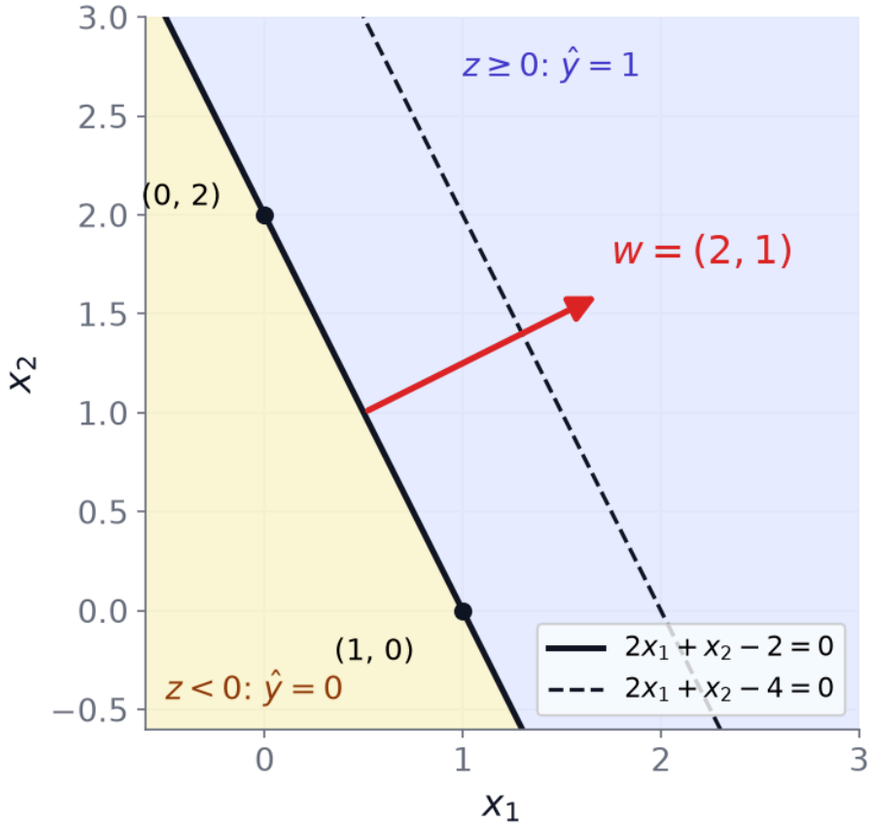
Geometry: the perceptron draws a line

The **decision boundary** is the set of points where the score is exactly 0:

$$w_1 x_1 + w_2 x_2 + b = 0$$

- In 2 dimensions it is a **line**; in n dimensions, a **hyperplane**.
- On one side $z \geq 0$ (class 1), on the other $z < 0$ (class 0).
- w points towards the class 1 side.

Example: $w = (2, 1)$, $b = -2$: the line $2x_1 + x_2 - 2 = 0$.



Why w is orthogonal to the line, and what b does

w is orthogonal

Take two points x_A and x_B on the line:

$$\begin{aligned} w^\top x_A + b &= 0 \\ w^\top x_B + b &= 0 \end{aligned} \implies w^\top (x_A - x_B) = 0$$

$x_A - x_B$ is a direction of the line, and its dot product with w is 0: w is orthogonal to the line.

b shifts the line

Intercepts with the axes (set the other coordinate to 0):

$$x_1 = -\frac{b}{w_1}\ (x_2 = 0), \qquad x_2 = -\frac{b}{w_2}\ (x_1 = 0)$$

- $b = 0$: the line passes through the origin.
- Same w, new b: a **parallel** line (dashed in the figure, $b = -4$).
- Example $w = (2, 1)$, $b = -2$: intercepts $(1, 0)$ and $(0, 2)$.

Exercise 2 · Draw the line of exercise 1

Exercise 8 min pen and paper

1. Write the equation of the AND line ($w = (1, 1)$, $b = -1.5$) and find its two intercepts with the axes.
2. Draw it with the 4 points of the AND table. Draw w. On which side is class 1?
3. Check that w is orthogonal to the line, using the direction vector $(1, -1)$.
4. Draw your OR line on the same figure. What changed, and what did not?

Your notes / answers (saved in this browser)

Learning: where do w and b come from?

We do not choose the weights by hand: the perceptron **learns** them from labelled examples (x, y).

1. Start with $w = 0$, $b = 0$.
2. Go through the examples one by one. For each one, predict $\hat{y}$ and compare it with y.
3. Update the parameters with the **perceptron rule**:

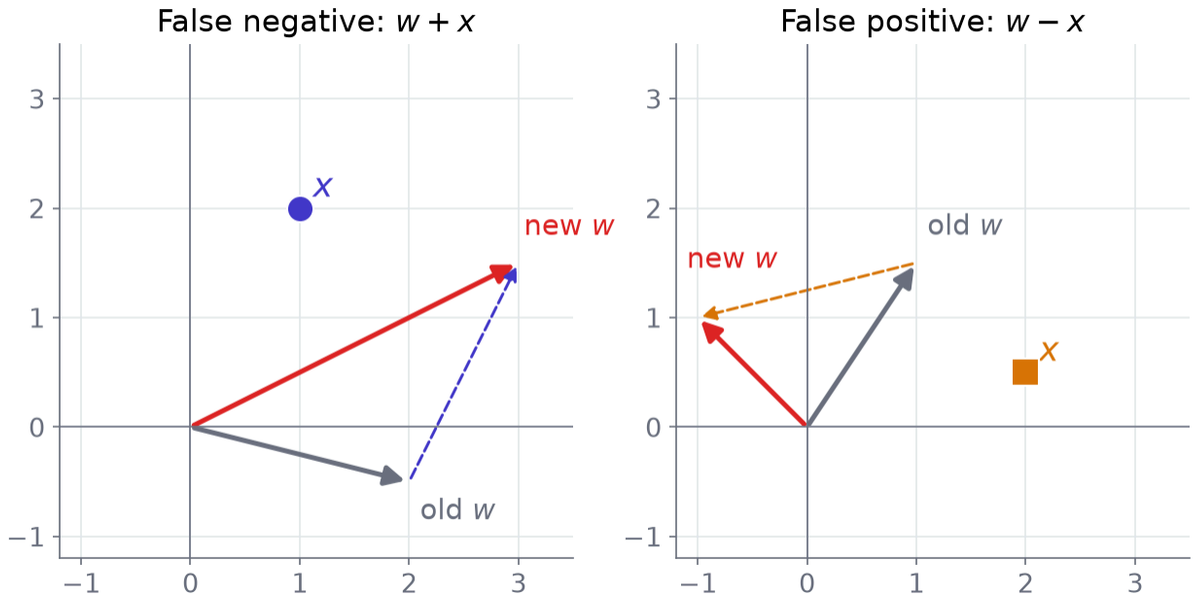
$$w \leftarrow w + \eta\,(y - \hat{y})\,x \qquad\qquad b \leftarrow b + \eta\,(y - \hat{y})$$

- $\eta > 0$ is the **learning rate** (today $\eta = 1$). The error $y - \hat{y}$ is -1, 0 or $+1$.
- One pass over the whole dataset is an **epoch**. Repeat epochs until one epoch has no mistake.

The three cases of the rule

case	$y - \hat{y}$	update of w
correct	0	nothing
false negative $y = 1, \hat{y} = 0$	$+1$	$w + \eta x$
false positive $y = 0, \hat{y} = 1$	-1	$w - \eta x$

- False negative: w turns **towards** x.
- False positive: w turns **away** from x.



Why the rule helps: one line of algebra

After a mistake on x, the new parameters are $w' = w + \eta e\, x$ and $b' = b + \eta e$, with $e = y - \hat{y}$. The new score of the same x:

$$\begin{aligned} z' &= w'^{\top} x + b' = (w + \eta e\, x)^{\top} x + b + \eta e \\ &= \underbrace{w^{\top} x + b}_{z} + \eta e\, (\underbrace{x^{\top} x}_{\|x\|^2} + 1) \\ &= z + \eta e\, (\|x\|^2 + 1) \end{aligned}$$

- $\|x\|^2 + 1 > 0$, so false negative ($e = +1$): the score **increases**; false positive ($e = -1$): it **decreases**.
- Each update moves the score of the misclassified point in the right direction (it may need several updates).

Exercise 3 · Train a perceptron by hand

point	x_1	x_2	y
A	1	3	1
B	1	0	0
C	3	1	1
D	0	0	0

Start: $w = (0, 0)$, $b = 0$, $\eta = 1$. Visit A, B, C, D in this order. Remember: $z = 0$ gives $\hat{y} = 1$.

Exercise 15 min pen and paper

1. Make a table with one row per visit: point, z, $\hat{y}$, error $y - \hat{y}$, new w, new b.
2. Run epochs until a whole epoch has no mistake. How many mistakes in each epoch?
3. Give the final line and draw it with the 4 points.

Your notes / answers (saved in this browser)

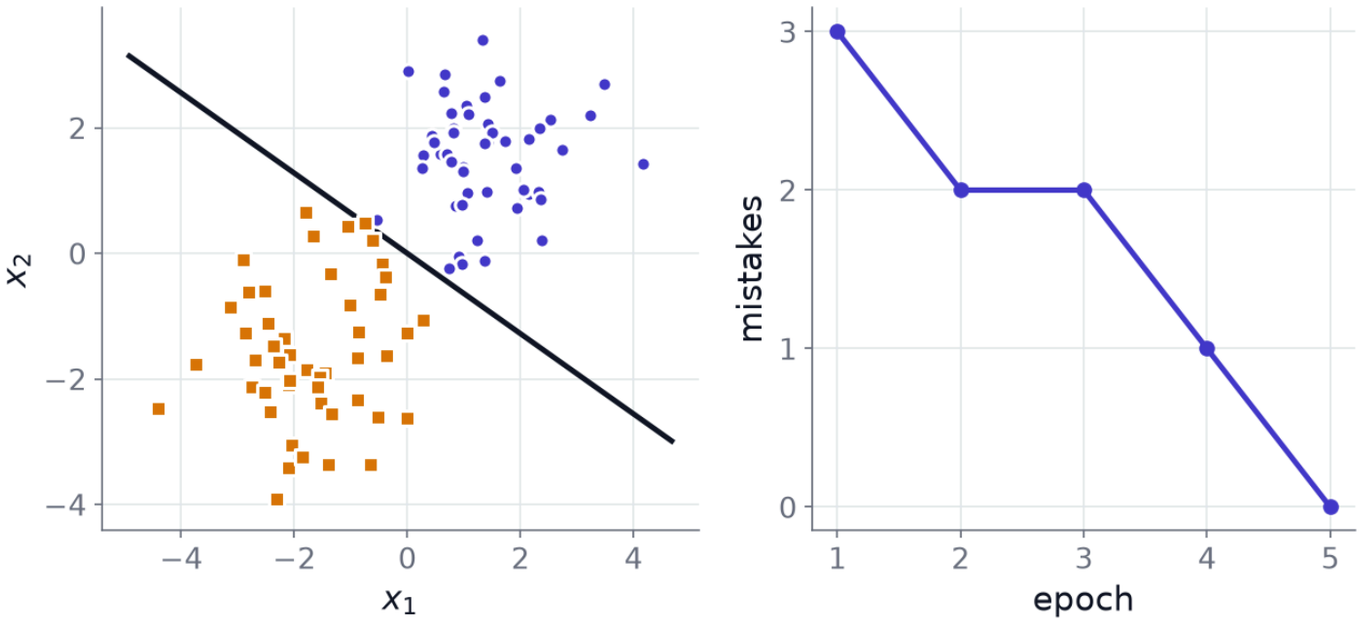
Does it always stop? The convergence theorem

Novikoff (1962)

If the data are **linearly separable** (a line separates the two classes), the perceptron rule makes a **finite number of mistakes**, then stops.

More precisely: if every $\|x\| \leq R$ and a separating line leaves a margin γ around it, the number of mistakes is at most $(R/\gamma)^2$. Well-separated classes: few mistakes.

The theorem says nothing when the data are **not** separable (see the limits).



Two separable clusters (TP Part C): the number of mistakes per epoch drops to 0.

Vectorised prediction: all examples at once

Put the m examples in the rows of a matrix X of shape (m, n). Then `X @ w` computes the m dot products $w^\top x^{(i)}$ in one operation:

$$\underbrace{X}_{(m,n)}\ \underbrace{w}_{(n,)} + b = \underbrace{z}_{(m,)}$$

```
import numpy as np
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])   # the AND inputs, shape (4, 2)
w, b = np.array([1, 1]), -1.5
z = X @ w + b                                    # [-1.5 -0.5 -0.5  0.5]
y_hat = (z ⩾ 0).astype(int)                      # [0 0 0 1]: booleans to 0/1
```

No Python loop over the examples: NumPy does it in compiled code, much faster for large m.

The perceptron in NumPy

```
def predict(X, w, b):
    return (X @ w + b ⩾ 0).astype(int)        # one x or a whole matrix X

def train_perceptron(X, y, eta=1.0, n_epochs=10):
    w, b = np.zeros(X.shape[1]), 0.0
    for epoch in range(n_epochs):
        mistakes = 0
        for x_i, y_i in zip(X, y):             # one example at a time
            error = y_i - predict(x_i, w, b)   # -1, 0 or +1
            w, b = w + eta * error * x_i, b + eta * error
            mistakes += int(error ≠ 0)
        if mistakes == 0:
            break                              # a whole epoch without mistake
    return w, b
```

On the 4 points of exercise 3, `train_perceptron(X, y)` returns `w = [1., 1.]`, `b = -2.0`: the result of the table.

Break: 10 minutes

TP · The perceptron in NumPy

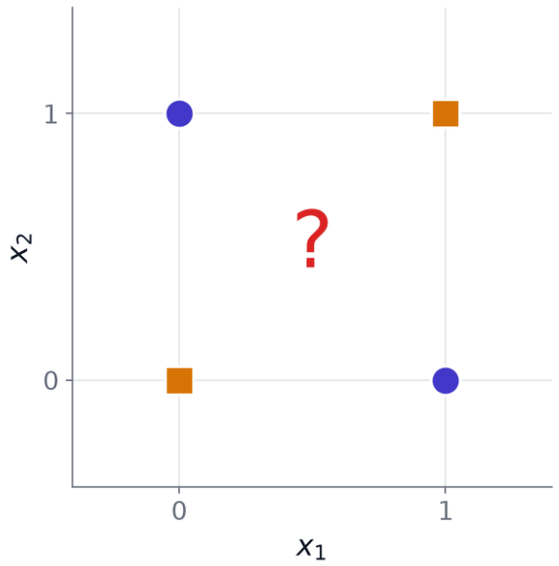
TP 55 min tp/TP01_perceptron_numpy.ipynb, Parts A to C

1. Part A: the step function, one prediction, vectorised predictions on the AND, OR and NAND tables.
2. Part B: the update rule, the training loop, and exercise 3 of the course reproduced exactly.
3. Part C: a 2-D dataset of two clusters, training, accuracy and the decision line.

Deliverable: every cell of Parts A to C prints "Exercise validated!".

Your notes / answers (saved in this browser)

Limit 1 · XOR: no line can do it



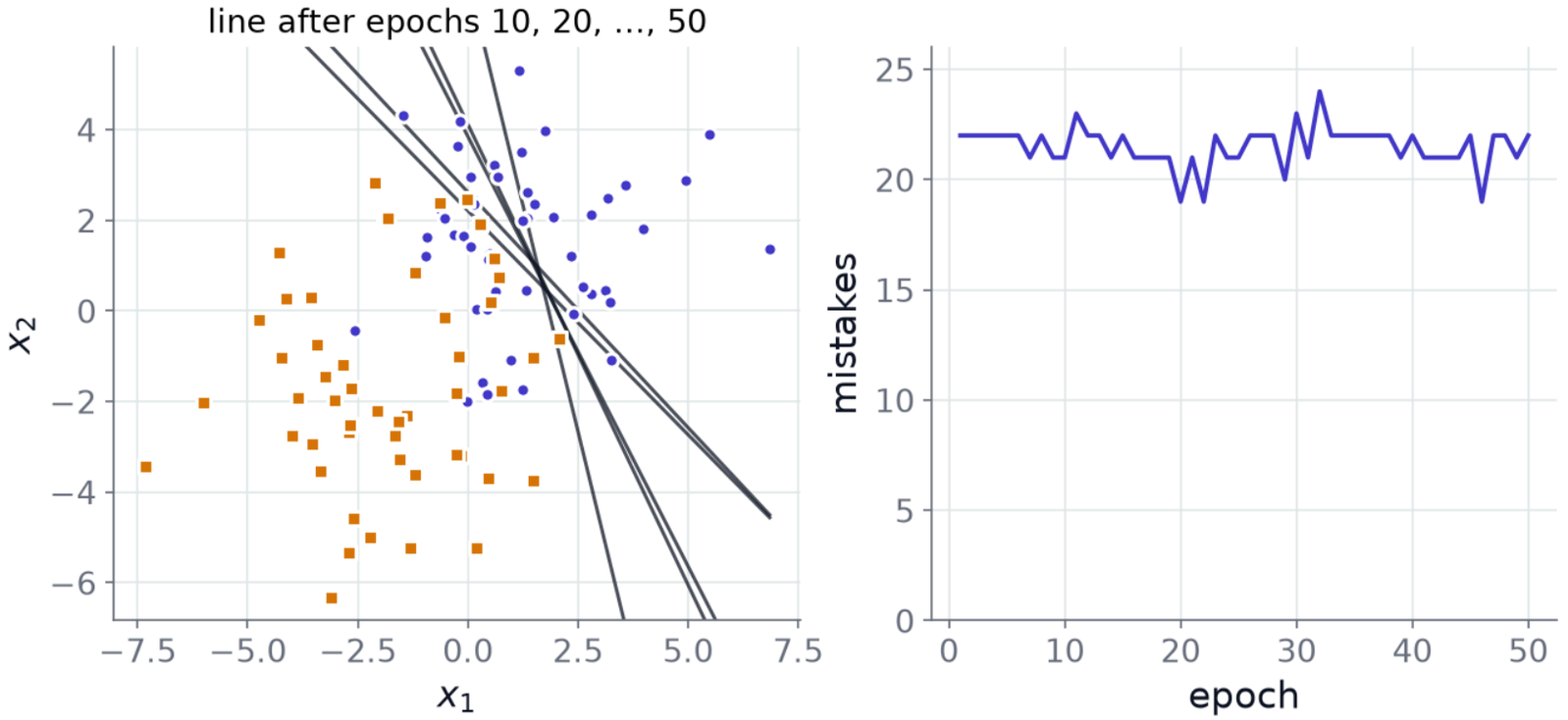
XOR: 1 when exactly one input is 1.

Exercise 4 8 min pen and paper

1. Write the 4 conditions on w_1, w_2, b that a perceptron computing XOR must satisfy (one per input, with ≥ 0 or < 0).
2. Add the two conditions of the inputs $(1, 0)$ and $(0, 1)$. Combine with the condition of $(0, 0)$.
3. Conclude: does such a perceptron exist?

Your notes / answers (saved in this browser)

Limit 2 · Non-separable data: the rule never stops



- When the clusters overlap, every epoch has mistakes: the rule keeps correcting, the line keeps jumping.
- The final line depends on when we stop. The rule has no notion of the "best" line: we would need a **quantity to minimise**.

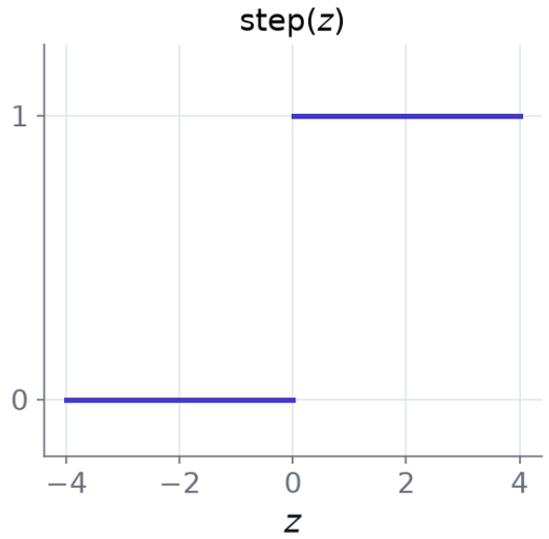
Limits 3 and 4 · No confidence, no gradient

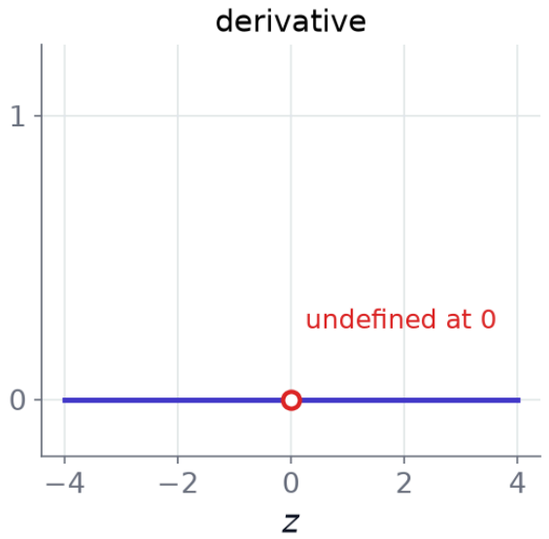
3. The output is 0 or 1

A point just above the line ($z = 0.01$) and a point far away ($z = 100$) both give $\hat{y} = 1$. No probability, no idea of how sure the model is.

4. The step has no useful derivative

Its derivative is 0 everywhere (undefined at 0). A small change of w does not change $\hat{y}$: no gradient, so **no gradient descent** and no way to train several layers.





From the limits to the next sessions

limit of the perceptron	the fix	when
output 0/1, no confidence	replace the step by the smooth **sigmoid**: a probability	Session 2
no derivative	the sigmoid is differentiable: **gradient descent**	Session 2
no "best" line on non-separable data	minimise a **loss** (the log loss)	Session 2
one line only (XOR)	several neurons in **layers**, trained by backpropagation	Session 3

Next session

Keep one neuron, replace the step by the sigmoid $\sigma(z) = 1/(1 + e^{-z})$: this is **logistic regression**, trained by gradient descent.

TP · Part D: the limits in code

TP 15 min tp/TP01_perceptron_numpy.ipynb, Part D

1. Train on XOR: the number of mistakes never reaches 0.
2. Train on overlapping clusters: plot the mistakes per epoch.
3. Check numerically that the step function has a zero slope; list the 4 limits.

Deliverable: Part D validated; fast students do the Going further part (a scikit-learn style class).

Your notes / answers (saved in this browser)

Summary

1. Perceptron: $z = w^\top x + b$, $\hat{y} = 1$ if $z \geq 0$, else 0.
2. It draws a line (a hyperplane): w is orthogonal to it, b shifts it.
3. Rule: $w \leftarrow w + \eta\,(y - \hat{y})\,x$, $b \leftarrow b + \eta\,(y - \hat{y})$; it stops on separable data (Novikoff).
4. Limits: XOR, non-separable data, no probability, no gradient.

Next session: replace the step by the sigmoid and train with gradient descent (logistic regression).

Project checkpoint: create your repository, choose 5 to 8 artwork classes and download the subset with `project/make_wikiart_subset.py`.